

Hackathon Web Transactionnelle

Plateforme de Gestion de Ligues Sportives Communautaires

Checkpoint de progression – Séance 08h00 à 11h00

Durée maximale du hackathon : 48 heures

Objectif de cette séance

Vous êtes déjà en plein hackathon. Cette séance n'est donc pas un cours magistral classique. Elle sert à vérifier votre avancement, clarifier votre MVP, débloquer les problèmes techniques et vous pousser vers une version fonctionnelle du projet.

Votre objectif n'est pas de tout faire. Votre objectif est de livrer une version utilisable, cohérente, testable et présentable de la plateforme.

Rappel du sujet

Vous devez développer une plateforme web moderne permettant de gérer des ligues sportives communautaires. L'application connecte principalement deux types d'utilisateurs :

- des **organiseurs**, qui créent et gèrent des tournois, des équipes, des demandes et des matchs ;
- des **joueurs**, qui recherchent des équipes, envoient des demandes d'adhésion et suivent leurs matchs.

L'idée générale est de créer une sorte de plateforme de rencontre sportive communautaire, comparable à un « Meetup » pour les sports amateurs.

1. Informations générales de l'équipe

Nom du projet	Sports League
Nom de l'équipe	Salle 4
Membres de l'équipe	Eugene, Sebastien Yves Mohamed Massinissa, Ziane
Répartition des rôles	Frontend : Massinissa Backend : Sebastien BDD / Prisma : Sebastien Auth / Stripe : Les deux
Lien GitHub	https://github.com/Tiger1804z/Hackathon-Sport.git

2. Stand-up rapide de début de séance

Chaque équipe doit répondre rapidement aux questions suivantes.

Question	Réponse de l'équipe
Qu'avez-vous fait hier entre 14h et 17h ?	Project init, commencer les actions, configuration et l'implémentation de Clerk, essayer de faire fonctionner le webhook clerk
Quelle partie du projet est déjà commencée ?	Les actions, Clerk, ui
Qu'est-ce qui bloque actuellement l'équipe ?	Clerk webhook pour synchroniser les users avec la db + leurs roles apres l'inscription
Quelle fonctionnalité voulez-vous rendre visible avant 11h ?	user sync and redirection

Règle du hackathon

Une fonctionnalité non visible, non testable ou non démontrable ne compte pas vraiment. À partir de maintenant, vous devez penser en termes de démonstration : qu'est-ce que vous pouvez réellement montrer ?

3. MVP obligatoire : checklist de progression

Cochez ce qui est déjà fait, puis indiquez ce qui reste à faire.

Fonctionnalité MVP	Statut	Notes / Problèmes / Prochaines étapes
Authentification avec Clerk	☒	Marche
Synchronisation User Clerk vers Prisma	☒	Clerk Webhook fonctionne
Gestion des rôles : PLAYER, ORGANIZER, ADMIN	☒	pas de role admin

Profil joueur : ville, sport, niveau, poste	☐	Juste une maquette
Dashboard organisateur	☒	manque juste Prisma
Création et gestion des tournois	☒	actions done
Création et gestion des équipes	☒	done
Recherche d'équipes côté joueur	☐	not implemented yet
Demandes d'adhésion	☒	done
Acceptation / refus des demandes	☒	done
Matches : création, liste, modification simple	☐	missing list
Païement Stripe pour tournoi payant	☒	done missing test
Section admin custom	☐	no

4. Priorité absolue : la fonctionnalité centrale

Fonctionnalité centrale du projet

La fonctionnalité la plus importante du sujet est le système de **demandes d'adhésion**. C'est elle qui donne une vraie logique métier à l'application.

Le scénario principal est le suivant :

1. Un organisateur crée un tournoi.
2. L'organisateur crée une ou plusieurs équipes dans ce tournoi.
3. Un joueur recherche une équipe disponible.
4. Le joueur envoie une demande d'adhésion.
5. Si le tournoi est payant, le joueur doit passer par Stripe.
6. L'organisateur voit les demandes reçues.
7. L'organisateur accepte ou refuse la demande.
8. Si la demande est acceptée, le joueur devient membre de l'équipe.

Votre état actuel sur cette fonctionnalité

Question	Réponse
Avez-vous déjà créé le modèle JoinRequest dans Prisma ?	oui
Avez-vous empêché les demandes en double ?	oui
Avez-vous vérifié la capacité maximale d'une équipe ?	oui
Avez-vous une Server Action createJoinRequest() ?	oui
Avez-vous une Server Action acceptJoinRequest() ?	oui
L'acceptation se fait-elle dans une transaction Prisma ?	oui
Le joueur voit-il le statut de ses demandes ?	backend est prêt

L'organisateur voit-il les demandes reçues ?	backend ready
--	---------------

5. Modèle de données : vérification rapide

Votre base de données doit au minimum contenir les éléments suivants.

- ☐ User
- ☐ PlayerProfile
- ☐ Tournament
- ☐ Team
- ☐ JoinRequest
- ☐ Match

État du schéma Prisma

Modèle	Créé ?	Notes
User	Oui	A REMPLIR
PlayerProfile	Oui	A REMPLIR
Tournament	Oui	A REMPLIR
Team	Oui	A REMPLIR
JoinRequest	Oui	A REMPLIR
Match	Oui	A REMPLIR

6. Pages et routes à viser

Page / Route	Rôle	Statut
/	Landing publique	done

/sign-in et /sign-up	Authentification Clerk	done
/profile	Profil joueur	done
/dashboard	Dashboard organisateur	done
/tournaments	Liste / gestion des tournois	done
/tournaments/[id]	Détail d'un tournoi	done
/teams	Recherche d'équipes	started
/teams/[id]	Détail d'une équipe	started
/my-requests	Demandes du joueur	started
/requests	Demandes reçues par l'organisateur	started
/matches	Matches	started
/admin	Administration custom	abandoned

7. Server Actions et Route Handlers

Server Actions prioritaires

- ☐ updatePlayerProfile()
- ☐ createTournament()
- ☐ updateTournament(id)
- ☐ deleteTournament(id)
- ☐ createTeam()
- ☐ createJoinRequest()
- ☐ cancelJoinRequest(id)
- ☐ acceptJoinRequest(id)

- ☐ rejectJoinRequest(id)
- ☐ createMatch()
- ☐ updateMatchScore(id)

Route Handlers nécessaires

- ☐ POST /api/webhooks/clerk
- ☐ POST /api/webhooks/stripe
- ☐ POST /api/checkout

8. Stripe : décision immédiate

Attention

Si votre application permet de créer un tournoi avec des frais d'inscription supérieurs à 0, le paiement Stripe devient obligatoire dans le MVP.

Si vous êtes en retard techniquement, commencez d'abord par faire fonctionner tout le parcours avec un tournoi gratuit. Ensuite seulement, ajoutez le flux Stripe pour les tournois payants.

Question	Réponse
Avez-vous prévu des tournois payants ?	Oui / Non / Pas encore décidé
Le champ entryFee existe-t-il dans Tournament ?	oui
Le champ paymentStatus existe-t-il dans JoinRequest ?	oui
La route /api/checkout est-elle commencée ?	oui
Le webhook Stripe est-il commencé ?	oui
Avez-vous testé Stripe CLI ?	non pas encore , il est installé le test est en cours

9. Objectif concret avant 11h00

Chaque équipe doit choisir une seule priorité principale pour la fin de la séance.

À 11h00, votre équipe doit pouvoir montrer au moins une chose concrète :

- une page fonctionnelle ;
- une connexion réussie ;
- une création de tournoi ;
- une création d'équipe ;
- une demande d'adhésion ;
- une acceptation de demande ;
- une liste affichée depuis Prisma ;
- ou une autre fonctionnalité visible et testable.

Élément	Réponse de l'équipe
Fonctionnalité à démontrer avant 11h00	Connexion réussie
Responsable principal de cette fonctionnalité	Partagé
Ce qui est déjà fait	Connexion avec Clerk + sync avec webhook + Role
Ce qui reste à faire	redirection vers les routes appropriées
Risque principal	aucun
Plan B si cela ne fonctionne pas	prioriser l'api pour le front-end puis attaquer les transactions

10. Priorisation : Must Have, Nice to Have, Abandon

Must Have

Ce qui doit absolument être fait pour que le projet soit présentable.

- Authentification avec Clerk, User sync, redirection basée sur le rôle
- CRUD des tournois/matches
- Pouvoir faire une transaction
- Navigation entre les pages

Nice to Have

Ce qui est intéressant seulement si le MVP fonctionne déjà.

- meilleur gui
- barre de recherches fonctionnelles
- Page d'admin custom

À abandonner si le temps manque

Ce qui peut être retiré sans détruire la logique principale du projet.

- A REMPLIR
- Page d'admin custom
- A REMPLIR

11. Pièges techniques à éviter

- Ne pas importer Prisma dans un Client Component.
- Ne pas oublier revalidatePath après une mutation.
- Ne jamais faire confiance aux données envoyées par le client.
- Toujours valider les entrées côté serveur avec Zod.
- Ne jamais exposer les secrets dans NEXT_PUBLIC_*.
- Ne pas vérifier l'auth uniquement dans le middleware : vérifier aussi dans les Server Actions sensibles.
- Ne pas passer trop de temps sur le design avant d'avoir une logique fonctionnelle.
- Ne pas commencer les bonus avant d'avoir terminé le MVP.

12. Mini pitch technique

À la fin de la séance, chaque équipe doit être capable de présenter rapidement son avancement.

Notre projet s'appelle : Sports League

Notre plateforme permet de : Connecter des joueurs et des organisateurs de ligues sportives.

Notre utilisateur joueur peut : Rejoindre des équipes et consulter des matchs.

Notre utilisateur organisateur peut : Créer et gérer des tournois.

Notre fonctionnalité centrale est : La gestion de tournois sportifs communautaires.

Notre base de données contient actuellement : Les utilisateurs, équipes, tournois et matchs.

La fonctionnalité que nous pouvons démontrer maintenant est : L'authentification et l'interface utilisateur.

Ce qui nous reste à faire en priorité est : Finaliser les webhooks et connecter les données réelles.

13. Feedback de l'enseignant

Critère	Feedback
Clarté de l'idée	A REMPLIR
Avancement technique	A REMPLIR
Qualité du modèle de données	A REMPLIR
Respect du MVP	A REMPLIR
Organisation de l'équipe	A REMPLIR
Prochaine action recommandée	A REMPLIR

14. Message final

Un hackathon ne se gagne pas avec la quantité d'idées. Il se gagne avec la capacité à choisir, construire, tester et présenter.

Un petit parcours complet qui fonctionne vaut mieux qu'une grande application incomplète.

Priorisez le scénario principal :

Organisateur crée un tournoi → crée une équipe → joueur demande à rejoindre → organisateur accepte → joueur rejoint l'équipe

Si ce scénario fonctionne, votre projet possède déjà un coeur solide.

Clarifiez. Priorisez. Codez. Testez.

Bon courage pour la suite du hackathon.